

# Evolution of Football Teams

Projekt Zespołowy (II stopień)

Sebastian Karlewicz, Łukasz Łukaszewski, Paweł Kwieciński,  
Grzegorz Febert, Miłosz Maculewicz

October 2025

## **Abstract**

This project studies how football (soccer) team compositions evolve over time by modeling player co-appearances as weighted graphs. We extract match and appearance data for selected top European clubs in 2014–2024 from a public dataset and generate two JSON files describing matches and involved players. For a chosen season represented by year, we build a network where nodes are players, and an edge weight equals the number of matches in which two players appear together. Finally, we provide an interactive Java application, utilizing GraphStream with Swing, which allows us to explore the network with filters by year, teams, players, and edge weights.

# Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Project goal and scope</b>                    | <b>2</b>  |
| 1.1      | Goal . . . . .                                   | 2         |
| 1.2      | Scope: clubs and time period . . . . .           | 2         |
| <b>2</b> | <b>Data sources</b>                              | <b>3</b>  |
| 2.1      | Primary dataset . . . . .                        | 3         |
| 2.2      | Access method . . . . .                          | 3         |
| <b>3</b> | <b>Data extraction and filtering</b>             | <b>4</b>  |
| 3.1      | Overview . . . . .                               | 4         |
| 3.2      | Filtering criteria . . . . .                     | 4         |
| 3.3      | Outputs produced by extraction . . . . .         | 4         |
| 3.3.1    | matches.json . . . . .                           | 4         |
| 3.3.2    | players.json . . . . .                           | 4         |
| <b>4</b> | <b>Data cleaning and modelling decisions</b>     | <b>5</b>  |
| 4.1      | Cleaning rules implemented . . . . .             | 5         |
| 4.2      | Model definition . . . . .                       | 5         |
| <b>5</b> | <b>Application architecture</b>                  | <b>6</b>  |
| 5.1      | Building blocks . . . . .                        | 6         |
| 5.2      | Communication between blocks . . . . .           | 6         |
| 5.3      | Workflow diagram . . . . .                       | 6         |
| <b>6</b> | <b>Visualization and interactive exploration</b> | <b>7</b>  |
| 6.1      | Libraries . . . . .                              | 7         |
| 6.2      | Interactive controls . . . . .                   | 7         |
| 6.3      | Graph construction (per selected year) . . . . . | 7         |
| <b>7</b> | <b>Application usage</b>                         | <b>8</b>  |
| 7.1      | Environment . . . . .                            | 8         |
| 7.2      | One-command script . . . . .                     | 8         |
| <b>8</b> | <b>Team organisation and contributions</b>       | <b>9</b>  |
| <b>A</b> | <b>Weekly minutes (annex)</b>                    | <b>10</b> |
| <b>A</b> | <b>AI prompts (annex)</b>                        | <b>11</b> |

# Chapter 1

## Project goal and scope

### 1.1 Goal

The goal is to collect and format data about the composition of the football team and represent the relationship between players as configurable graphs. Vertices correspond to players, and two players are connected if they both appeared in the same match. The link is weighted by the number of matches they share, not differentiating whether playing in the same team or against each other.

### 1.2 Scope: clubs and time period

We focus on matches between a fixed set of clubs (e.g., FC Barcelona, Real Madrid, Manchester City, Manchester United, Bayern Munich, Liverpool, Arsenal, Chelsea, Juventus, AC Milan, Inter Milan, PSG, Atletico Madrid, Borussia Dortmund) in the date range 2014-01-01 to 2024-12-31, as implemented in `extract_games.py`.

## Chapter 2

# Data sources

### 2.1 Primary dataset

We use the Kaggle *Football Data dataset from Transfermarkt* (Kaggle: `davidcariboo/player-scores`).

[1] The extraction pipeline uses three CSV tables from the dataset: `games.csv`, `appearances.csv`, and `players.csv`.

### 2.2 Access method

Data are loaded programmatically via `kagglehub` using the `KaggleDatasetAdapter.PANDAS` adapter, which provides Pandas DataFrames for further processing.

## Chapter 3

# Data extraction and filtering

### 3.1 Overview

The extraction stage is implemented in `extract_games.py`. Its responsibility is to:

- load raw tables (`games`, `appearances`, `players`),
- filter matches to the target set of clubs and time range,
- build two compact outputs: `matches.json` and `players.json`.

### 3.2 Filtering criteria

A match is included if:

- both `home_club_id` and `away_club_id` belong to the predefined set of club IDs,
- its date is in the interval 2014–2024 inclusively.

Dates are parsed using `pd.to_datetime(..., errors='coerce')`. Invalid dates become `NaT` and are then excluded by the range filter.

### 3.3 Outputs produced by extraction

#### 3.3.1 `matches.json`

The file is a JSON dictionary indexed by `game_id`. Each entry has:

- `date`: ISO string YYYY-MM-DD or null,
- `home_team`: {`club_id`, `club_name`, `players`[]},
- `away_team`: {`club_id`, `club_name`, `players`[]},

where `players` is a list of integer `player_id` values that appeared for the given club in this match.

#### 3.3.2 `players.json`

The file is a JSON dictionary mapping player IDs to their names:

```
"<player_id>": "<player_name>"
```

Keys are stored as strings to comply with JSON object key constraints.

## Chapter 4

# Data cleaning and modelling decisions

### 4.1 Cleaning rules implemented

Our first cleaning step is to ensure that identifiers are consistent and that missing values are removed before producing outputs:

- in appearances: `player_id` is required; missing values are dropped,
- IDs are cast to integers to avoid mixed types across joins/filters,
- in players table: duplicates are removed by `drop_duplicates(subset=['player_id'])`,
- for each match and club we keep unique player IDs (`unique().tolist()`).

### 4.2 Model definition

For a fixed year  $y$ , we define a weighted undirected graph  $G_y = (V_y, E_y, w)$ :

- $V_y$ : set of players who appear in at least one selected match in year  $y$ ,
- $(u, v) \in E_y$  if players  $u$  and  $v$  appear in the same match, either in the same team or in opposing teams,
- $w(u, v)$ : number of matches in year  $y$  where  $u$  and  $v$  co-appear.

# Chapter 5

## Application architecture

### 5.1 Building blocks

The repository implements the following blocks and data flow:

1. **Extraction (Python):** `extract_games.py` loads raw data set tables and produces `matches.json` and `players.json`.
2. **Visualization (Java):** `FootballNetworkGraph.java` loads `matches.json` and `players.json`, constructs  $G_y$  on demand, and renders it using `GraphStream`. [2]

### 5.2 Communication between blocks

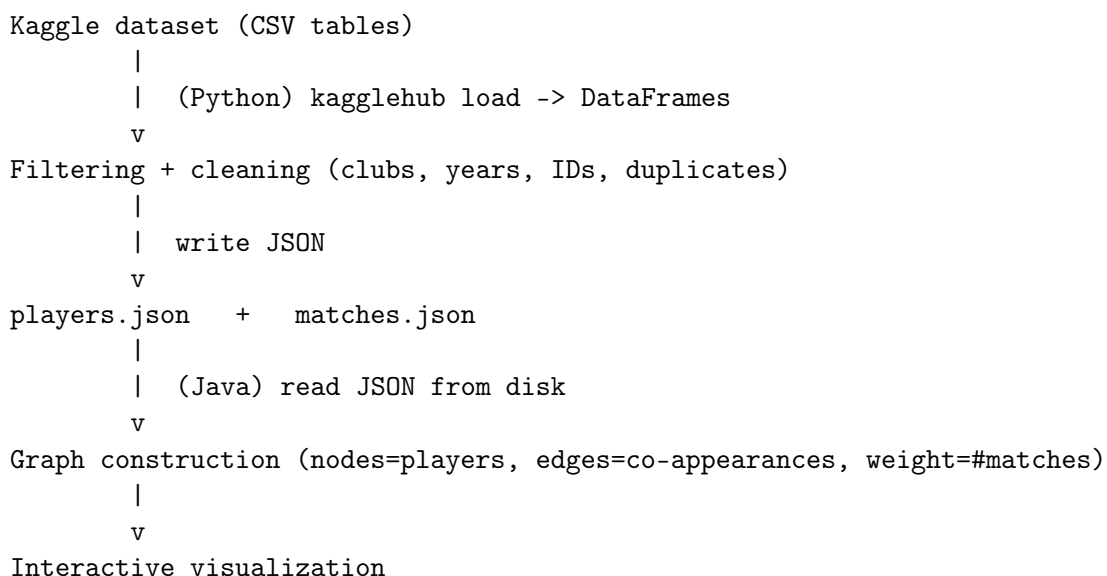
The blocks communicate strictly via files:

- Python outputs: `matches.json`, `players.json`,
- Java input: reads `matches.json` and `players.json` from disk at startup, then rebuilds the graph in memory when filters change.

This design makes the pipeline easy to reproduce and independent of runtime services.

### 5.3 Workflow diagram

The complete workflow can be summarized as follows.





## Chapter 6

# Visualization and interactive exploration

### 6.1 Libraries

Graph visualization is implemented with the GraphStream 2.0 core and user interface modules - Swing, JavaFX. [2]

### 6.2 Interactive controls

The application provides:

- year slider (select season/year),
- weight range sliders (filter edges by number of matches played together),
- team selection list (include/exclude clubs),
- player selection list (include/exclude individual nodes),
- option to show isolated nodes (players without edges after filtering).

### 6.3 Graph construction (per selected year)

For the selected year, the program:

- scans all matches in `matches.json` for that year,
- collects players from selected clubs and selected players,
- counts co-appearances for each unordered pair, producing weights,
- materialize nodes and edges in GraphStream and assign UI labels and styles.

# Chapter 7

## Application usage

### 7.1 Environment

Python dependencies are listed in `requirements.txt`. The Java compilation uses the JARs in `lib/`.

### 7.2 One-command script

We provide `create_and_run.sh` that:

- creates a Python virtual environment,
- installs requirements,
- runs `extract_games.py` to generate JSON files,
- compiles `FootballNetworkGraph.java` with `lib/*`,
- runs the Java application.

## Chapter 8

# Team organisation and contributions

- Sebastian Karlewicz: open and maintain the code repository and grant instructor access.
- Miłosz Maculewicz: maintain the report in  $\text{\LaTeX}$  and ensure Overleaf access is granted to the instructor.
- All members: contribute to code, data validation, report writing, and final presentation.

## Appendix A

### Weekly minutes (annex)

The team followed a meeting scheme of planning and coding sessions 19:00–20:00 each Saturday.

## Appendix A

### AI prompts (annex)

- Create a plan on implementing football team composition, propose stack that would be optimal with keeping in mind that visual representation is required in GraphStream. Split work into 3-4 roles
- Propose a source of legitimate data about football team composition that can be easily aquired using Python
- Create an example of loading JSON data in Java

# Bibliography

- [1] *Football data from transfermarkt (kaggle dataset: Davidcariboo/player-scores)*, <https://www.kaggle.com/datasets/davidcariboo/player-scores>, Accessed: 2026-01-30.
- [2] *Graphstream gs-core (github repository)*, <https://github.com/graphstream/gs-core>, Accessed: 2026-01-04.